

```

# =====
# DJANGO CODE: library_project/settings.py
# =====
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = 'django-insecure-library-management-secret-key-bca-project-2024'
DEBUG = True
ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'library',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'library_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'library_project.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',

```

```

        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'Asia/Kolkata'
USE_I18N = True
USE_TZ = True
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
LOGIN_URL = '/login/'
LOGIN_REDIRECT_URL = '/dashboard/'
LOGOUT_REDIRECT_URL = '/login/'

# =====
# DJANGO CODE: library/models.py
# =====
from django.db import models

class Book(models.Model):
    """Model representing a book in the library."""
    CATEGORY_CHOICES = [
        ('Fiction', 'Fiction'),
        ('Non-Fiction', 'Non-Fiction'),
        ('Science', 'Science'),
        ('Technology', 'Technology'),
        ('History', 'History'),
        ('Mathematics', 'Mathematics'),
        ('Literature', 'Literature'),
        ('Philosophy', 'Philosophy'),
        ('Biography', 'Biography'),
        ('Self-Help', 'Self-Help'),
    ]

    name = models.CharField(max_length=255)
    author = models.CharField(max_length=255)
    category = models.CharField(max_length=50, choices=CATEGORY_CHOICES,
default='Fiction')
    quantity = models.PositiveIntegerField(default=1)
    available = models.PositiveIntegerField(default=1)
    added_date = models.DateField(auto_now_add=True)

    class Meta:
        ordering = ['name']

    def __str__(self):
        return f"{self.name} by {self.author}"

    @property
    def is_available(self):
        return self.available > 0

```

```

class Student(models.Model):
    """Model representing a student."""
    COURSE_CHOICES = [
        ('BCA', 'BCA'),
        ('BBA', 'BBA'),
        ('B.Com', 'B.Com'),
        ('B.Sc', 'B.Sc'),
        ('B.Tech', 'B.Tech'),
        ('MBA', 'MBA'),
        ('MCA', 'MCA'),
        ('M.Sc', 'M.Sc'),
    ]

    name = models.CharField(max_length=255)
    email = models.EmailField(unique=True)
    course = models.CharField(max_length=50, choices=COURSE_CHOICES,
default='BCA')
    phone = models.CharField(max_length=15, blank=True, null=True)
    enrollment_date = models.DateField(auto_now_add=True)

    class Meta:
        ordering = ['name']

    def __str__(self):
        return f"{self.name} ({self.course})"

    @property
    def issued_books_count(self):
        return self.issuedbook_set.filter(returned=False).count()

class IssuedBook(models.Model):
    """Model representing a book issued to a student."""
    book = models.ForeignKey(Book, on_delete=models.CASCADE)
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    issue_date = models.DateField(auto_now_add=True)
    due_date = models.DateField()
    return_date = models.DateField(null=True, blank=True)
    returned = models.BooleanField(default=False)

    class Meta:
        ordering = ['-issue_date']

    def __str__(self):
        return f"{self.book.name} -> {self.student.name}"

    @property
    def is_overdue(self):
        from datetime import date
        if not self.returned:
            return date.today() > self.due_date
        return False

```

```

# =====
# DJANGO CODE: library/views.py
# =====
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.db.models import Q
from datetime import date, timedelta
from .models import Book, Student, IssuedBook

def login_view(request):
    """Admin login page."""
    if request.user.is_authenticated:
        return redirect('dashboard')
    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
        password = request.POST.get('password', '').strip()
        user = authenticate(request, username=username, password=password)
        if user is not None and user.is_staff:
            login(request, user)
            messages.success(request, f'Welcome back, {user.first_name or user.username}!')
            return redirect('dashboard')
        else:
            messages.error(request, 'Invalid credentials or insufficient permissions.')
            return render(request, 'library/login.html')

def logout_view(request):
    """Admin logout."""
    logout(request)
    messages.info(request, 'You have been logged out.')
    return redirect('login')

@login_required
def dashboard(request):
    """Main dashboard with summary stats."""
    total_books = Book.objects.count()
    total_students = Student.objects.count()
    issued_books = IssuedBook.objects.filter(returned=False).count()
    overdue_books = sum(1 for ib in IssuedBook.objects.filter(returned=False) if ib.is_overdue)
    recent_issues = IssuedBook.objects.filter(returned=False).select_related('book', 'student')[:5]

    context = {
        'total_books': total_books,
        'total_students': total_students,
        'issued_books': issued_books,
        'overdue_books': overdue_books,
        'recent_issues': recent_issues,
    }

```

```

    }
    return render(request, 'library/dashboard.html', context)

@login_required
def book_list(request):
    """List and search books."""
    query = request.GET.get('q', '').strip()
    books = Book.objects.all()
    if query:
        books = books.filter(
            Q(name__icontains=query) |
            Q(author__icontains=query) |
            Q(category__icontains=query)
        )
    return render(request, 'library/books.html', {'books': books, 'query':
query})

@login_required
def add_book(request):
    """Add a new book."""
    if request.method == 'POST':
        name = request.POST.get('name', '').strip()
        author = request.POST.get('author', '').strip()
        category = request.POST.get('category', 'Fiction').strip()
        quantity = int(request.POST.get('quantity', 1))

        if not name or not author:
            messages.error(request, 'Book name and author are required.')
            return redirect('add_book')

        Book.objects.create(
            name=name,
            author=author,
            category=category,
            quantity=quantity,
            available=quantity
        )
        messages.success(request, f'Book "{name}" added successfully!')
        return redirect('book_list')

    return render(request, 'library/add_book.html', {'categories':
Book.CATEGORY_CHOICES})

@login_required
def edit_book(request, pk):
    """Edit an existing book."""
    book = get_object_or_404(Book, pk=pk)

    if request.method == 'POST':
        book.name = request.POST.get('name', '').strip()
        book.author = request.POST.get('author', '').strip()
        book.category = request.POST.get('category', book.category).strip()
        quantity = int(request.POST.get('quantity', book.quantity))
        diff = quantity - book.quantity

```

```

        book.quantity = quantity
        book.available = max(0, book.available + diff)
        book.save()
        messages.success(request, f'Book "{book.name}" updated successfully!')
        return redirect('book_list')

    return render(request, 'library/edit_book.html', {
        'book': book,
        'categories': Book.CATEGORY_CHOICES
    })

```

```

@login_required
def delete_book(request, pk):
    """Delete a book."""
    book = get_object_or_404(Book, pk=pk)
    if request.method == 'POST':
        name = book.name
        book.delete()
        messages.success(request, f'Book "{name}" deleted successfully!')
    return redirect('book_list')

```

```

@login_required
def student_list(request):
    """List and search students."""
    query = request.GET.get('q', '').strip()
    students = Student.objects.all()
    if query:
        students = students.filter(
            Q(name__icontains=query) |
            Q(email__icontains=query) |
            Q(course__icontains=query)
        )
    return render(request, 'library/students.html', {'students': students,
        'query': query})

```

```

@login_required
def add_student(request):
    """Add a new student."""
    if request.method == 'POST':
        name = request.POST.get('name', '').strip()
        email = request.POST.get('email', '').strip()
        course = request.POST.get('course', 'BCA').strip()
        phone = request.POST.get('phone', '').strip()

        if not name or not email:
            messages.error(request, 'Name and email are required.')
            return redirect('add_student')

        if Student.objects.filter(email=email).exists():
            messages.error(request, 'A student with this email already exists.')
            return redirect('add_student')

        Student.objects.create(name=name, email=email, course=course,

```

```

phone=phone)
    messages.success(request, f'Student "{name}" added successfully!')
    return redirect('student_list')

    return render(request, 'library/add_student.html', {'courses':
Student.COURSE_CHOICES})

@login_required
def edit_student(request, pk):
    """Edit an existing student."""
    student = get_object_or_404(Student, pk=pk)

    if request.method == 'POST':
        student.name = request.POST.get('name', '').strip()
        student.email = request.POST.get('email', '').strip()
        student.course = request.POST.get('course', student.course).strip()
        student.phone = request.POST.get('phone', '').strip()
        student.save()
        messages.success(request, f'Student "{student.name}" updated
successfully!')
        return redirect('student_list')

    return render(request, 'library/edit_student.html', {
        'student': student,
        'courses': Student.COURSE_CHOICES
    })

@login_required
def delete_student(request, pk):
    """Delete a student."""
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        name = student.name
        student.delete()
        messages.success(request, f'Student "{name}" deleted successfully!')
    return redirect('student_list')

@login_required
def issued_books(request):
    """List all issued books."""
    issues = IssuedBook.objects.filter(returned=False).select_related('book',
'student')
    return render(request, 'library/issued_books.html', {'issues': issues})

@login_required
def issue_book(request):
    """Issue a book to a student."""
    books = Book.objects.filter(available__gt=0)
    students = Student.objects.all()

    if request.method == 'POST':
        book_id = request.POST.get('book')
        student_id = request.POST.get('student')

```

```

        due_days = int(request.POST.get('due_days', 14))

        book = get_object_or_404(Book, pk=book_id)
        student = get_object_or_404(Student, pk=student_id)

        if book.available <= 0:
            messages.error(request, 'This book is not available.')
            return redirect('issue_book')

        if IssuedBook.objects.filter(book=book, student=student,
returned=False).exists():
            messages.error(request, f'{student.name} already has "{book.name}"
issued.')
            return redirect('issue_book')

        IssuedBook.objects.create(
            book=book,
            student=student,
            due_date=date.today() + timedelta(days=due_days)
        )
        book.available -= 1
        book.save()
        messages.success(request, f'"{book.name}" issued to {student.name}
successfully!')
        return redirect('issued_books')

    return render(request, 'library/issue_book.html', {'books': books,
'students': students})

@login_required
def return_book(request, pk):
    """Return an issued book."""
    issued = get_object_or_404(IssuedBook, pk=pk, returned=False)
    if request.method == 'POST':
        issued.returned = True
        issued.return_date = date.today()
        issued.save()
        issued.book.available += 1
        issued.book.save()
        messages.success(request, f'"{issued.book.name}" returned
successfully!')
        return redirect('issued_books')

@login_required
def return_history(request):
    """Show all returned books."""
    history = IssuedBook.objects.filter(returned=True).select_related('book',
'student')
    return render(request, 'library/return_history.html', {'history': history})

# =====
# DJANGO CODE: library/urls.py
# =====

```



```

from django.urls import path
from . import views

urlpatterns = [
    # Auth
    path('', views.login_view, name='home'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),

    # Dashboard
    path('dashboard/', views.dashboard, name='dashboard'),

    # Books
    path('books/', views.book_list, name='book_list'),
    path('books/add/', views.add_book, name='add_book'),
    path('books/edit/<int:pk>/', views.edit_book, name='edit_book'),
    path('books/delete/<int:pk>/', views.delete_book, name='delete_book'),

    # Students
    path('students/', views.student_list, name='student_list'),
    path('students/add/', views.add_student, name='add_student'),
    path('students/edit/<int:pk>/', views.edit_student, name='edit_student'),
    path('students/delete/<int:pk>/', views.delete_student,
name='delete_student'),

    # Issue & Return
    path('issued/', views.issued_books, name='issued_books'),
    path('issue/', views.issue_book, name='issue_book'),
    path('return/<int:pk>/', views.return_book, name='return_book'),
    path('history/', views.return_history, name='return_history'),
]

# =====
# HTML CONTENT: templates/library/base.html
# =====
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}LibraryMS{% endblock %} – Library Management
System</title>
    <meta name="description" content="College Library Management System – Manage
books, students, and issue records easily.">
    {% load static %}
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
    <link rel="stylesheet" href="{% static 'css/sidebar.css' %}">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&di
splay=swap" rel="stylesheet">
    <link rel="stylesheet"

```

```

href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css"
>
</head>
<body>

{% if user.is_authenticated %}
<!-- Sidebar -->
<aside class="sidebar" id="sidebar">
    <div class="sidebar-header">
        <div class="sidebar-logo">
            <i class="fa-solid fa-book-open-reader"></i>
        </div>
        <div class="sidebar-brand">
            <span class="brand-name">LibraryMS</span>
            <span class="brand-sub">Management System</span>
        </div>
        <button class="sidebar-toggle" id="sidebarToggle" aria-label="Toggle
sidebar">
            <i class="fa-solid fa-bars"></i>
        </button>
    </div>

    <nav class="sidebar-nav">
        <div class="nav-group">
            <span class="nav-label">Main</span>
            <a href="{% url 'dashboard' %}" class="nav-item">
                <i class="fa-solid fa-gauge-high"></i>
                <span>Dashboard</span>
            </a>
        </div>

        <div class="nav-group">
            <span class="nav-label">Library</span>
            <a href="{% url 'book_list' %}" class="nav-item">
                <i class="fa-solid fa-book"></i>
                <span>Books</span>
            </a>
            <a href="{% url 'student_list' %}" class="nav-item">
                <i class="fa-solid fa-user-graduate"></i>
                <span>Students</span>
            </a>
        </div>

        <div class="nav-group">
            <span class="nav-label">Transactions</span>
            <a href="{% url 'issued_books' %}" class="nav-item">
                <i class="fa-solid fa-book-open"></i>
                <span>Issued Books</span>
            </a>
            <a href="{% url 'issue_book' %}" class="nav-item"
style="padding-left:2.5rem">
                <i class="fa-solid fa-plus"></i>
                <span>Issue Book</span>
            </a>

```

```

        <a href="{% url 'return_history' %}" class="nav-item">
            <i class="fa-solid fa-clock-rotate-left"></i>
            <span>Return History</span>
        </a>
    </div>
</nav>

<div class="sidebar-footer">
    <div class="user-info">
        <div class="user-avatar">{{ user.username|first|upper }}</div>
        <div>
            <p class="user-name">{{ user.get_full_name|default:user.username
}}</p>
            <p class="user-role">Administrator</p>
        </div>
    </div>
    <a href="{% url 'logout' %}" class="logout-btn" title="Logout">
        <i class="fa-solid fa-right-from-bracket"></i>
    </a>
</div>
</aside>

<!-- Main Wrapper -->
<div class="main-wrapper" id="mainWrapper">
    <header class="top-bar">
        <button class="mobile-menu-btn" id="mobileMenuBtn" aria-label="Open
menu">
            <i class="fa-solid fa-bars"></i>
        </button>
        <div class="page-title">
            <h1>{% block page_title %}Dashboard{% endblock %}</h1>
        </div>
        <div class="top-bar-actions">
            <span class="top-bar-user"><i class="fa-regular fa-user"></i> {{
user.username }}</span>
        </div>
    </header>

    {% if messages %}
    <div class="messages-container" id="messagesContainer">
        {% for message in messages %}
        <div class="alert alert-{{ message.tags }} fade-in" role="alert">
            <i class="fa-solid fa-circle-info"></i>
            {{ message }}
            <button class="alert-close"
onclick="this.parentElement.remove()">x</button>
        </div>
        {% endfor %}
    </div>
    {% endif %}

    <main class="main-content">
        {% block content %}{% endblock %}
    </main>

```

```
</div>
```

```
{% else %}  
{% block login_content %}{% endblock %}  
{% endif %}
```

```
<script src="{% static 'js/main.js' %}"></script>  
{% block extra_js %}{% endblock %}  
</body>  
</html>
```

```
# =====  
# HTML CONTENT: templates/library/dashboard.html  
# =====  
{% extends 'library/base.html' %}  
{% block title %}Dashboard{% endblock %}  
{% block page_title %}Dashboard{% endblock %}
```

```
{% block content %}  
<!-- Stats Cards -->  
<div class="stats-grid">  
    <div class="stat-card stat-card--blue">  
        <div class="stat-card-icon">  
            <i class="fa-solid fa-book"></i>  
        </div>  
        <div class="stat-card-body">  
            <p class="stat-label">Total Books</p>  
            <h2 class="stat-value" id="countBooks">{{ total_books }}</h2>  
            <p class="stat-sub">In collection</p>  
        </div>  
    </div>  
  
    <div class="stat-card stat-card--green">  
        <div class="stat-card-icon">  
            <i class="fa-solid fa-user-graduate"></i>  
        </div>  
        <div class="stat-card-body">  
            <p class="stat-label">Total Students</p>  
            <h2 class="stat-value" id="countStudents">{{ total_students }}</h2>  
            <p class="stat-sub">Registered</p>  
        </div>  
    </div>  
  
    <div class="stat-card stat-card--orange">  
        <div class="stat-card-icon">  
            <i class="fa-solid fa-book-open"></i>  
        </div>  
        <div class="stat-card-body">  
            <p class="stat-label">Issued Books</p>  
            <h2 class="stat-value" id="countIssued">{{ issued_books }}</h2>  
            <p class="stat-sub">Currently out</p>  
        </div>  
    </div>  
</div>
```

```

<div class="stat-card stat-card--red">
  <div class="stat-card-icon">
    <i class="fa-solid fa-triangle-exclamation"></i>
  </div>
  <div class="stat-card-body">
    <p class="stat-label">Overdue Books</p>
    <h2 class="stat-value" id="countOverdue">{{ overdue_books }}</h2>
    <p class="stat-sub">Past due date</p>
  </div>
</div>
</div>

```

```

<!-- Quick Actions -->
<div class="section-header">
  <h2 class="section-title">Quick Actions</h2>
</div>
<div class="quick-actions">
  <a href="{% url 'add_book' %}" class="quick-action-btn">
    <i class="fa-solid fa-plus"></i>
    <span>Add Book</span>
  </a>
  <a href="{% url 'add_student' %}" class="quick-action-btn">
    <i class="fa-solid fa-user-plus"></i>
    <span>Add Student</span>
  </a>
  <a href="{% url 'issue_book' %}" class="quick-action-btn">
    <i class="fa-solid fa-book-open"></i>
    <span>Issue Book</span>
  </a>
  <a href="{% url 'issued_books' %}" class="quick-action-btn">
    <i class="fa-solid fa-list-check"></i>
    <span>View Issued</span>
  </a>
</div>

```

```

<!-- Recent Issues -->
<div class="section-header">
  <h2 class="section-title">Recently Issued Books</h2>
  <a href="{% url 'issued_books' %}" class="section-link">View all <i
class="fa-solid fa-arrow-right"></i></a>
</div>

```

```

<div class="card">
  {% if recent_issues %}
  <div class="table-wrapper">
    <table class="table" id="recentIssuesTable">
      <thead>
        <tr>
          <th>#</th>
          <th>Book</th>
          <th>Student</th>
          <th>Course</th>
          <th>Issue Date</th>

```

```

        <th>Due Date</th>
        <th>Status</th>
    </tr>
</thead>
<tbody>
    {% for issue in recent_issues %}
    <tr>
        <td>{{ forloop.counter }}</td>
        <td>
            <div class="book-cell">
                <div class="book-icon-sm"><i class="fa-solid
fa-book"></i></div>
                <div>
                    <p class="book-name-sm">{{ issue.book.name
}}</p>
                    <p class="book-author-sm">{{ issue.book.author
}}</p>
                </div>
            </div>
        </td>
        <td>{{ issue.student.name }}</td>
        <td><span class="badge badge--blue">{{ issue.student.course
}}</span></td>
        <td>{{ issue.issue_date|date:"d M Y" }}</td>
        <td>{{ issue.due_date|date:"d M Y" }}</td>
        <td>
            {% if issue.is_overdue %}
                <span class="badge badge--red"><i class="fa-solid
fa-circle-exclamation"></i> Overdue</span>
            {% else %}
                <span class="badge badge--green"><i class="fa-solid
fa-circle-check"></i> Active</span>
            {% endif %}
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>
</div>
{% else %}
<div class="empty-state">
    <i class="fa-solid fa-book-open empty-icon"></i>
    <p>No books currently issued.</p>
    <a href="{% url 'issue_book' %}" class="btn btn-primary">Issue a
Book</a>
</div>
{% endif %}
</div>
{% endblock %}

```

```

# =====
# HTML CONTENT: templates/library/books.html
# =====

```

```

{% extends 'library/base.html' %}
{% block title %}Books{% endblock %}
{% block page_title %}Book Management{% endblock %}

{% block content %}
<div class="page-toolbar">
    <div class="search-box">
        <i class="fa-solid fa-magnifying-glass search-icon"></i>
        <input type="text" id="searchInput" class="search-input"
placeholder="Search books..." value="{{ query }}" autocomplete="off">
    </div>
    <a href="{% url 'add_book' %}" class="btn btn-primary">
        <i class="fa-solid fa-plus"></i> Add Book
    </a>
</div>

<div class="card">
    {% if books %}
    <div class="table-wrapper">
        <table class="table" id="booksTable">
            <thead>
                <tr>
                    <th>#</th>
                    <th>Book Name</th>
                    <th>Author</th>
                    <th>Category</th>
                    <th>Qty</th>
                    <th>Available</th>
                    <th>Added On</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                {% for book in books %}
                <tr>
                    <td>{{ forloop.counter }}</td>
                    <td>
                        <div class="book-cell">
                            <div class="book-icon-sm"><i class="fa-solid
fa-book"></i></div>
                            <span class="book-name-sm">{{ book.name }}</span>
                        </div>
                    </td>
                    <td>{{ book.author }}</td>
                    <td><span class="badge badge--purple">{{ book.category
}}</span></td>
                    <td>{{ book.quantity }}</td>
                    <td>
                        {% if book.available > 0 %}
                        <span class="badge badge--green">{{ book.available }}
left</span>
                        {% else %}
                        <span class="badge badge--red">Out of stock</span>
                        {% endif %}
                </td>
                </tr>
            </tbody>
        </table>
    </div>
    {% endif %}

```

```

        </td>
        <td>{{ book.added_date|date:"d M Y" }}</td>
        <td>
            <div class="action-btns">
                <a href="{% url 'edit_book' book.pk %}"
class="btn-icon btn-icon--edit" title="Edit">
                    <i class="fa-solid fa-pen-to-square"></i>
                </a>
                <button class="btn-icon btn-icon--delete"
title="Delete" onclick="confirmDelete('{% url 'delete_book' book.pk %}', '{{
book.name|escapejs }}')">
                    <i class="fa-solid fa-trash"></i>
                </button>
            </div>
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>
</div>
{% endif %}
</div>
{% endblock %}

```

```

# =====
# HTML CONTENT: templates/library/students.html
# =====
{% extends 'library/base.html' %}
{% block title %}Students{% endblock %}
{% block page_title %}Student Management{% endblock %}

{% block content %}
<div class="page-toolbar">
    <div class="search-box">
        <i class="fa-solid fa-magnifying-glass search-icon"></i>
        <input type="text" id="searchInput" class="search-input"
placeholder="Search students..." value="{{ query }}" autocomplete="off">
    </div>
    <a href="{% url 'add_student' %}" class="btn btn-primary">
        <i class="fa-solid fa-user-plus"></i> Add Student
    </a>
</div>

<div class="card">
    {% if students %}
    <div class="table-wrapper">
        <table class="table" id="studentsTable">
            <thead>
                <tr>
                    <th>#</th>
                    <th>Student Name</th>
                    <th>Email</th>
                    <th>Course</th>

```



```

        <th>Phone</th>
        <th>Enrolled On</th>
        <th>Books Issued</th>
        <th>Actions</th>
    </tr>
</thead>
<tbody>
    {% for student in students %}
    <tr>
        <td>{{ forloop.counter }}</td>
        <td>
            <div class="student-cell">
                <div class="student-avatar">{{
student.name|first|upper }}</div>
                <span>{{ student.name }}</span>
            </div>
        </td>
        <td>{{ student.email }}</td>
        <td><span class="badge badge--blue">{{ student.course
}}</span></td>
        <td>{{ student.phone|default:"-"}</td>
        <td>{{ student.enrollment_date|date:"d M Y"}</td>
        <td>
            {% if student.issued_books_count > 0 %}
            <span class="badge badge--orange">{{
student.issued_books_count }} books</span>
            {% else %}
            <span class="text-muted">None</span>
            {% endif %}
        </td>
        <td>
            <div class="action-btns">
                <a href="{% url 'edit_student' student.pk %}"
class="btn-icon btn-icon--edit" title="Edit">
                    <i class="fa-solid fa-pen-to-square"></i>
                </a>
                <button class="btn-icon btn-icon--delete"
title="Delete" onclick="confirmDelete('{% url 'delete_student' student.pk %}',
'{{ student.name|escapejs }}')">
                    <i class="fa-solid fa-trash"></i>
                </button>
            </div>
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>
</div>
{% endif %}
</div>
{% endblock %}

```

=====

CSS CONTENT: static/css/style.css

=====

```
:root {
  --sidebar-width: 260px;
  --sidebar-collapsed: 70px;
  --topbar-height: 65px;
  --primary: #4f46e5;
  --primary-light: #6366f1;
  --primary-dark: #3730a3;
  --success: #10b981;
  --warning: #f59e0b;
  --danger: #ef4444;
  --info: #3b82f6;
  --bg: #f1f5f9;
  --surface: #ffffff;
  --sidebar-bg: #1e1b4b;
  --sidebar-text: #c7d2fe;
  --sidebar-active: #4f46e5;
  --text: #1e293b;
  --text-muted: #94a3b8;
  --border: #e2e8f0;
  --radius: 12px;
  --radius-sm: 8px;
  --shadow: 0 1px 3px rgba(0,0,0,.08), 0 4px 16px rgba(0,0,0,.06);
  --shadow-lg: 0 10px 40px rgba(0,0,0,.12);
  --font: 'Inter', system-ui, sans-serif;
  --transition: 0.22s ease;
}

*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: var(--font); background: var(--bg); color: var(--text);
min-height: 100vh; }
a { text-decoration: none; color: inherit; }
button { cursor: pointer; border: none; background: none; font-family: inherit;
}
input, select, textarea { font-family: inherit; }

/* Sidebar */
.sidebar {
  position: fixed; top: 0; left: 0;
  width: var(--sidebar-width); height: 100vh;
  background: var(--sidebar-bg);
  display: flex; flex-direction: column;
  z-index: 100;
  transition: width var(--transition);
  overflow: hidden;
}
.sidebar-header {
  display: flex; align-items: center; gap: 12px;
  padding: 20px 16px;
  border-bottom: 1px solid rgba(255,255,255,.08);
  min-height: 72px;
}
.sidebar-logo {
```

```

width: 40px; height: 40px; flex-shrink: 0;
background: var(--primary);
border-radius: 10px;
display: flex; align-items: center; justify-content: center;
color: #fff; font-size: 18px;
}
.sidebar-brand { flex: 1; overflow: hidden; }
.brand-name { display: block; color: #fff; font-weight: 700; font-size: 15px;
white-space: nowrap; }
.brand-sub { display: block; color: var(--sidebar-text); font-size: 11px;
white-space: nowrap; opacity: .7; }
.sidebar-toggle { color: var(--sidebar-text); font-size: 18px; padding: 6px;
border-radius: 6px; transition: background var(--transition); flex-shrink: 0; }
.sidebar-toggle:hover { background: rgba(255,255,255,.1); }

.sidebar-nav { flex: 1; overflow-y: auto; padding: 12px 0; }
.nav-group { margin-bottom: 8px; }
.nav-label { display: block; padding: 8px 20px 4px; font-size: 10px;
font-weight: 600; letter-spacing: .1em; color: rgba(199,210,254,.45);
text-transform: uppercase; white-space: nowrap; overflow: hidden; }
.nav-item { display: flex; align-items: center; gap: 12px; padding: 10px 20px;
color: var(--sidebar-text); border-radius: 8px; margin: 2px 10px; transition:
all var(--transition); white-space: nowrap; font-size: 14px; font-weight: 500; }
.nav-item i { font-size: 16px; width: 20px; text-align: center; flex-shrink: 0;
}
.nav-item:hover { background: rgba(255,255,255,.08); color: #fff; }
.nav-item.active { background: var(--primary); color: #fff; }

.sidebar-footer { padding: 16px; border-top: 1px solid rgba(255,255,255,.08);
display: flex; align-items: center; gap: 10px; }
.user-info { display: flex; align-items: center; gap: 10px; flex: 1; overflow:
hidden; }
.user-avatar { width: 36px; height: 36px; flex-shrink: 0; background:
var(--primary); border-radius: 50%; display: flex; align-items: center;
justify-content: center; color: #fff; font-weight: 700; font-size: 14px; }
.user-name { font-size: 13px; font-weight: 600; color: #fff; white-space:
nowrap; overflow: hidden; text-overflow: ellipsis; }
.user-role { font-size: 11px; color: var(--sidebar-text); opacity: .7; }
.logout-btn { color: var(--sidebar-text); font-size: 16px; padding: 8px;
border-radius: 8px; transition: all var(--transition); flex-shrink: 0; }
.logout-btn:hover { background: rgba(239,68,68,.2); color: #f87171; }

/* Main Wrapper */
.main-wrapper { margin-left: var(--sidebar-width); min-height: 100vh;
transition: margin-left var(--transition); display: flex; flex-direction:
column; }
.top-bar { height: var(--topbar-height); background: var(--surface);
border-bottom: 1px solid var(--border); display: flex; align-items: center; gap:
16px; padding: 0 28px; position: sticky; top: 0; z-index: 50; }
.top-bar h1 { font-size: 20px; font-weight: 700; }
.mobile-menu-btn { display: none; font-size: 20px; color: var(--text); padding:
6px; }
.top-bar-actions { margin-left: auto; }
.top-bar-user { font-size: 13px; color: var(--text-muted); }

```

```

.top-bar-user i { margin-right: 6px; }
.page-title { flex: 1; }
.main-content { flex: 1; padding: 28px; max-width: 1400px; width: 100%; }

/* Cards & Stats */
.card { background: var(--surface); border-radius: var(--radius); box-shadow:
var(--shadow); overflow: hidden; }
.stats-grid { display: grid; grid-template-columns: repeat(auto-fit,
minmax(220px, 1fr)); gap: 20px; margin-bottom: 28px; }
.stat-card { background: var(--surface); border-radius: var(--radius); padding:
24px; display: flex; align-items: center; gap: 18px; box-shadow: var(--shadow);
transition: transform var(--transition), box-shadow var(--transition);
border-left: 4px solid transparent; }
.stat-card:hover { transform: translateY(-2px); box-shadow: var(--shadow-lg); }
.stat-card--blue { border-left-color: var(--info); }
.stat-card--green { border-left-color: var(--success); }
.stat-card--orange { border-left-color: var(--warning); }
.stat-card--red { border-left-color: var(--danger); }
.stat-card-icon { width: 52px; height: 52px; border-radius: 12px; display: flex;
align-items: center; justify-content: center; font-size: 22px; flex-shrink: 0; }
.stat-card--blue .stat-card-icon { background: #eff6ff; color: var(--info); }
.stat-card--green .stat-card-icon { background: #f0fdf4; color: var(--success);
}
.stat-card--orange.stat-card-icon { background: #fffbeb; color: var(--warning);
}
.stat-card--red .stat-card-icon { background: #fef2f2; color: var(--danger); }
.stat-label { font-size: 12px; font-weight: 600; color: var(--text-muted);
text-transform: uppercase; letter-spacing: .05em; }
.stat-value { font-size: 32px; font-weight: 800; line-height: 1.1; margin: 4px
0; }
.stat-sub { font-size: 12px; color: var(--text-muted); }

/* Buttons & Forms */
.btn { display: inline-flex; align-items: center; gap: 8px; padding: 10px 20px;
border-radius: var(--radius-sm); font-size: 14px; font-weight: 600; transition:
all var(--transition); white-space: nowrap; }
.btn-primary { background: var(--primary); color: #fff; }
.btn-primary:hover { background: var(--primary-dark); transform:
translateY(-1px); }
.btn-secondary { background: var(--bg); color: var(--text); border: 1px solid
var(--border); }
.btn-danger { background: var(--danger); color: #fff; }
.action-btns { display: flex; gap: 6px; }
.btn-icon { width: 32px; height: 32px; border-radius: 6px; display: flex;
align-items: center; justify-content: center; font-size: 13px; transition: all
var(--transition); }
.btn-icon--edit { background: #eff6ff; color: var(--info); }
.btn-icon--edit:hover { background: var(--info); color: #fff; }
.btn-icon--delete { background: #fef2f2; color: var(--danger); }
.btn-icon--delete:hover { background: var(--danger); color: #fff; }

/* Tables */
.table-wrapper { overflow-x: auto; }
.table { width: 100%; border-collapse: collapse; font-size: 14px; }

```

```

.table thead th { padding: 12px 16px; text-align: left; font-size: 12px;
font-weight: 700; text-transform: uppercase; letter-spacing: .06em; color:
var(--text-muted); background: #f8fafc; border-bottom: 1px solid var(--border);
}
.table tbody td { padding: 13px 16px; border-bottom: 1px solid var(--border);
vertical-align: middle; }
.table tbody tr:hover { background: #f8fafc; }
.badge { display: inline-flex; align-items: center; gap: 4px; padding: 3px 10px;
border-radius: 20px; font-size: 12px; font-weight: 600; }
.badge--blue { background: #eff6ff; color: #1d4ed8; }
.badge--green { background: #f0fdf4; color: #15803d; }
.badge--red { background: #fef2f2; color: #b91c1c; }
.badge--orange { background: #fffbeb; color: #b45309; }
.badge--purple { background: #f5f3ff; color: #6d28d9; }

/* Form Elements */
.search-input { width: 100%; padding: 10px 40px 10px 40px; border: 1px solid
var(--border); border-radius: var(--radius-sm); font-size: 14px; background:
var(--surface); transition: border-color var(--transition), box-shadow
var(--transition); }
.search-input:focus { outline: none; border-color: var(--primary); box-shadow: 0
0 3px rgba(79,70,229,.12); }

# =====
# JS CONTENT: static/js/main.js
# =====
const sidebar      = document.getElementById('sidebar');
const mainWrapper  = document.getElementById('mainWrapper');
const sidebarToggle = document.getElementById('sidebarToggle');
const mobileMenuBtn = document.getElementById('mobileMenuBtn');

if (sidebarToggle) {
  sidebarToggle.addEventListener('click', function () {
    if (window.innerWidth > 1024) {
      sidebar.classList.toggle('collapsed');
      if (sidebar.classList.contains('collapsed')) {
        mainWrapper.style.marginLeft = 'var(--sidebar-collapsed)';
      } else {
        mainWrapper.style.marginLeft = 'var(--sidebar-width)';
      }
    } else {
      sidebar.classList.toggle('open');
    }
  });
}

if (mobileMenuBtn) {
  mobileMenuBtn.addEventListener('click', function () {
    sidebar.classList.toggle('open');
  });
}

document.addEventListener('click', function (e) {

```

```

    if (window.innerWidth <= 1024 && sidebar &&
    sidebar.classList.contains('open')) {
        if (!sidebar.contains(e.target) && e.target !== mobileMenuBtn) {
            sidebar.classList.remove('open');
        }
    }
});

function confirmDelete(actionUrl, itemName) {
    const modal = document.getElementById('deleteModal');
    const form = document.getElementById('deleteForm');
    const message = document.getElementById('modalMessage');

    if (!modal || !form) return;
    form.action = actionUrl;
    message.textContent = `Are you sure you want to delete "${itemName}"? This
action cannot be undone.`;
    modal.classList.add('active');
    modal.querySelector('.btn-secondary').focus();
}

function closeDeleteModal() {
    const modal = document.getElementById('deleteModal');
    if (modal) modal.classList.remove('active');
}

document.addEventListener('click', function (e) {
    const modal = document.getElementById('deleteModal');
    if (modal && e.target === modal) closeDeleteModal();
});

document.addEventListener('keydown', function (e) {
    if (e.key === 'Escape') closeDeleteModal();
});

(function () {
    const alerts = document.querySelectorAll('.alert');
    alerts.forEach(function (alert) {
        setTimeout(function () {
            alert.style.transition = 'opacity 0.4s ease, transform 0.4s ease';
            alert.style.opacity = '0';
            alert.style.transform = 'translateY(-6px)';
            setTimeout(function () { alert.remove(); }, 400);
        }, 4000);
    });
})();

function animateCounter(el, target, duration) {
    let start = 0;
    const step = Math.ceil(target / (duration / 16));
    const timer = setInterval(function () {
        start += step;
        if (start >= target) { start = target; clearInterval(timer); }
        el.textContent = start;
    }, 250);
}

```

```
    }, 16);  
}  
  
(function () {  
  const counters = document.querySelectorAll('.stat-value');  
  counters.forEach(function (el) {  
    const val = parseInt(el.textContent, 10);  
    if (!isNaN(val) && val > 0) {  
      el.textContent = '0';  
      animateCounter(el, val, 700);  
    }  
  });  
})();  
})();
```